

Contents

Basic Information

Contents.....	1
Recent Updates:.....	2
Overview:.....	4
Installation:	5
HEPMC3:	7
Input:.....	8
DPMJET:	15
Output.....	17
Class Diagram.....	19
File Descriptions.....	20

Recent Updates:

Vxxx (Oct. 24, 2025) The Phi and K*0 are added in starlight+dpmjet output and their decay daughter are removed (dpmjet/dpmjetint.f). Proper switch (on/off) are added in slight.in.dpmjet to implement this. A proper boost is also applied to Phi and K*0 inside src/starlightdpmjet.cpp from particle CMS frame to DPMJET lab frame(in default case, inside dpmjet boost is only giving to stable particle).

vxxx (Nov. 13, 2024) Removed warning that $\omega \rightarrow \pi^+\pi^-\pi^0$ is unstable, since it seems OK.

vxxx (Nov. 10, 2023) Added Option for photoproduction of phi + direct KK, just like rho + direct pipi. This is particle type 933. The phi:direct KK ratio default is the same as for the rho + direct pipi

v328 - Implemented the HEPMC3 standard for two-photon interactions and coherent photonuclear interactions with INTERFERENCE turned off and 0n0n breakup modes. Made changes to the parameterization of the normalization constant for the default nuclear density distribution (2-parameter Fermi Model) in STARlight.

v327 - Updated CMAkeLists.tx and cmake_modules/FindDMJet.cmake with versions from Luan Arbeletche that accommodate the new version of DPMJET. Updated documentation to point to that version. n. b. There may still be a problem with running DPMJet, but this is much closer

v326 - Updated the Documentations to reflect recent changes.

v325 - Moved the repository from HEPForge to GIT. Hosted remotely on GitHub. Created a README.md and an AGPL LICENSE file for the repository.

GIT

v320 - Added channel: $\varphi \rightarrow e^+e^-e^+e^-$. with code (333011) - Nov 9 2022

v319 - updated the nuclear model for oxygen to the three parameter Fermi Model. Updated the code for Eta cut application to work correctly in assymetric collisions. Added PT_CUT and Eta_Cut functionalities for the FOURPRONG, ZOVERZ03, F2, F2Prime, and AXION Channels. Fixed the errors and bugs in the output of $\omega \rightarrow \pi^+\pi^-\pi^0$ Channel - Eta cuts are now applied only to the charged pions and not on the uncharged pion. Fixed the bug causing incomplete output EVENTS

in the FOURPRONG and Omega3pion channel. Fixed other bugs and potential bugs. Created copy constructors to fix warnings on new gcc compilers.

v317 - remove register keyword from randomgenerator.cpp for compatibility with C++17

v316 - added destructors for randomnumbergenerator and paramterbase, for LHCb

v315: added documentation for PRINTVM=2 command and a note about the last line of slight.in not working.

v313: Found a bug in channel: $\omega \rightarrow \pi^+\pi^-\pi^0$. Reconstructed ω mass is about 0.4% too high. While debugging, resolved an unrelated error. Marked $\omega \rightarrow \pi^+\pi^-\pi^0$ as UNSTABLE in this document.

v311: (UNSTABLE) Added channel: $\omega \rightarrow \pi^+\pi^-\pi^0$ with code (223211111).

v309: Added channels: $\rho^0 \rightarrow e^+e^-$ and $\rho^0 \rightarrow \mu^+\mu^-$ with codes(113011) and (113013) respectively.

v308: Added an option to display a header in the output file. This header will show input parameters in a similar format to eSTARlight. Option is controlled by input parameter "OUTPUT_HEADER", see below.

v307: Fixed 4-prong mass spectrum, properly converting dsigma/ds to dsigma/dW. The net effect is to scale the mass spectrum by $1/M_{\{4\pi\}}$, reducing the number of high-mass states.

v306: Updated gammavm.cpp, to properly output 4 pions with net charge 0. Also changed default Wmax for 4-prong final state to be the larger of the kinematic limit or 10 GeV; previously, it was unduly large for the LHC.

v305: Changed coefficient in calculation of pt2 in gammavm.cpp from 8 to 32. This changes the maximum p_T for vector mesons for calculations without interference from about 250 MeV/c to about 1 GeV/c. In the long run, we could rename INT_PT_MAX and make it required parameter with or without interference. This could make the running a bit more efficient.

v304: Fixed a bug in gammaaluminosity.cpp lines 404, where photonDensity was called with its arguments reversed. This bug affected the p_T spectrum when interference is turned on.

v299: Added hard-coded Woods-Saxon radii, thickness and density for ^{96}Ru and ^{96}Zr , for the RHIC isobar run. Data is from arXiv:1607.04697

v297: Changed normalized for Woods-Saxon density for non-predefined (i. e. not gold, lead, xenon or copper or nuclei with $Z < 7$) so that the density is properly normalized $\int d^3r \rho(r) = A$. The normalization was previously done for a hard-sphere nucleus, so this over-estimated the cross-sections by 5-10%.

v295: Added hard-coded values for xenon-129 to match the recent LHC run. Radius=5.36 fm, density=0.18406

v293: Introduced shared random number generator which can be externally passed by the user. All particle constants (masses, widths, branching ratios, and spins) can now also be set by the user, but should be changed from the default values with care.

v290: Added an new BREAKUP_MODE option to generate two-photon events in peripheral collisions. BREAKUP_MODE=8 sets a fixed impact parameter range, regardless of the presence of nuclear breakup; it is intended to study two-photon production in peripheral collisions. It requires two additional otherwise optional input lines, BMIN and BMAX, to set the impact parameter range. It does not (yet?) work for photonuclear interactions.

278: Added two new optional parameters:

IMPULSE_VM Normally 0, but can be set to 1 to perform an impulse approximation calculation (i.e. ignoring nuclear effects)

QUANTUM_GLAUBER. When set to 1, performs a quantum Glauber calculation, rather than a classical one. This leads to greatly increased rho and omega cross-sections for heavy nuclei, little effect for heavier mesons.

Also added a final state, 4432212, for J/psi -> pbar p

v276: Added two new optional parameters (BSLOPE_DEFINITION and BSLOPE_VALUE) for the p_T spectrum ('bslope') for proton targets or incoherent production on nuclei

v275: Added $\gamma\gamma$ to axion channel as two-photon channel 88, per S. Knapen et al., arXiv:1607.07083 v273: "Baseline" version, described in arXiv:1607.03838)

Overview:

The STARlight Monte Carlo models 2-photon and photon-Pomeron interactions in ultra-peripheral heavy ion collisions. The physics approach for the photon-Pomeron interactions is described in Klein and Nystrand, Phys. Rev. C60, 014903 (1999), with the p_T spectrum (including vector meson interference) discussed in Phys. Rev. Lett. 84, 2330 (2000). The 2-photon interactions are described in Baltz, Gorbunov, Klein, Nystrand, Phys.Rev. C80 044902 (2009).

STARlight has several input files, all of which are expected to be in the same directory as the starlight code. User-specified input parameters are read from a file named "slight.in"; these parameters are

described below in . Note that STARlight ignores the last line of slight.in, so it should be something irrelevant.

The simulated events are written to an ASCII file named "slight.out", which is described below in .

Installation:

GitHub Based Installation:

As of present, STARlight has been moved from HEPforge to GitHub. The detailed installation instructions for the latest version on GitHub can be found in the README.md file via this link <https://github.com/STARlightsim/STARlight/blob/main/README.md>. Please note that instructions on enabling either [PYTHIA](#) or [DPMJET](#) is still the same for both HEPforge and GitHub, so consult this material for those portions of the installation instructions.

Hepforge Based Installation:

The portion below may not be relevant anymore. This is because STARlight has been moved from HEPforge to GitHub with all its history. So you can easily find any previous version you might be interested in, in the GitHub repository. The HEPforge version is no longer maintained nor updated, and though the HEPforge repository has not yet been closed, this installation process cannot be guaranteed to work properly any longer.

To install & run STARlight in a *nix based environment, follow these steps(README):

Download the starlight package from 'Downloads' on the left sidebar of the homepage. The version in the example might be outdated.

```
-wget 'https://starlight.hepforge.org/downloads?f=starlight_r300.tar'
-mv 'downloads?f=starlight_r300.tar' starlight_r300.tar
-tar xvf starlight_r300.tar
```

Alternatively, one may obtain the latest version via svn. HEPforge uses phabricator and no longer allows for anonymous checkouts of the repository. (Please read <https://www.hepforge.org/guide.pdf> .)

To obtain an account, register here: <https://www.hepforge.org/register>

Once you are registered, login:

<https://phab.hepforge.org/auth/start/?next=%2F>

Set up a version control settings (VCS) password under your account's Settings->AUTHENTICATION->VCS password . The VCS password is needed to checkout the code.

(For remote users) To identify yourself, upload a SSH public key under account's Settings->AUTHENTICATION->SSH Public Keys with the button SSH Key Actions->Upload Public Key. This key will provide your identity when checking out the code as VCS. If you do not have a public ssh key to

upload, you may generate a pair on the same SSH Public Keys page with the button SSH Key Actions-> Generate Keypair.

With the private ssh key loaded on your machine (and public on their machine), use svn to checkout the trunk/:

```
-svn co svn+ssh://vcs@phab.hepforge.org/source/starlightsvn/trunk
```

Change to the installation directory of your choice

```
-mkdir /home/my/installation/dir
```

```
-cd /home/my/installation/dir
```

Setup the compilation with cmake

```
-cmake /path/to/trunk
```

Compile with (g)make

```
-gmake
```

Setup the input file, slight.in, for your simulation needs

```
-cp /path/to/trunk/config/slight.in .
```

```
-vim slight.in
```

Run

```
../starlight >& output.txt&
```

For more information and special scenarios, such as running with PYTHIA or DPMJET, consult the README files located in trunk/

If you would like to browse the code, please visit:

<https://phab.hepforge.org/source/starlightsvn/>

-----Before HEPForge updated their repository management system-----
To obtain the latest version:

```
-svn co
```

Alternatively:

```
-Visit
```

```
-Download the trunk [click on the download symbol in the Size column]
```

```
-Unpackage the zip file. The trunk/ represents <PathToSource>
```

To build Starlight:

```
- First create your build directory <BUILDDIR> (e.g. mkdir bin)
```

```
- $ cd <BUILDDIR>
```

```
- $ cmake <PathToSource>
```

```
- $ make
```

This creates an executable file, starlight, in the build directory.

To clean the build:

```
- $ make clean
```

To run starlight, a configuration file, slight.in, is needed. Examples of slight.in may be found in the config/ directory.

To run:

```
$ ./starlight
```

Enabling Pythia:

To simulate the γ , γ' , and e channels, you need Pythia v8.2 or higher to handle their decays. To enable Pythia support you need to run cmake with the option `-DENABLE_PYTHIA=ON` and have `$PYTHIADIR` pointing to the top directory of Pythia8. [Note: when building Pythia, be sure to enable shared libraries(.so). `./configure --enable-shared` before compiling Pythia.]

```
$ setenv PYTHIADIR /my/local/pythia8
```

```
$ cmake <PathToSource> -DENABLE_PYTHIA=ON
```

Note: v8.2+ is necessary since the Pythia directory structure changed[trunk/cmake_modules/FindPythia8.cmake depends on the structure layout], liblhpdfdummy was removed, and Standalone:allowResDec was removed.

To enable DPMJET, please see the passage on

HEPMC3:

The HEPMC3 standard is the latest standard for writing event records or output files in high energy physics event generators. STARlight has implemented this new standard by interfacing with the CERN HEPMC3 module. But it is important to note the following as there are situations where STARlight reverts to the old output standard.

The HEPMC3 standard is implemented for both two-photon and photonuclear interactions. But in photonuclear interactions, it is not implemented for incoherent interaction, neither is it implemented for coherent interactions when INTERFERENCE is turned on. So in these cases, STARlight reverts to the old output standard.

The HEPMC3 standard has only been implemented for the breakup-mode where there is no coulomb or hadronic breakup. This is `BREAKUP_MODE = 4, 0n0n`. In all other breakup modes, STARlight reverts to the old standard.

How to turn-on the HEPMC3 feature.

When the HEPMC3 feature is turned off (the default situation), STARlight's output follows the old standard and is written out to the `slight.out` file. To turn on the HEPMC3 feature, the CERN HEPMC3 module needs to be installed and linked to STARlight, when

STARlight's code is being compiled. With this successfully done, STARlight provides two outputs, the slight.hepmc file (in hepMC3 format) and the slight.out file (in the old STARlight format) .

An extended version of the old STARlight output that contains the momenta and energy of the outgoing beams, as an addition to those of the outgoing final state particles, can also be accessed (with/without installing the CERN HEPMC3 module) in the slight.out file, by setting the HEPMC3_EXTENDED_OUTPUT parameter to 1 in the input slight.in file.

The instructions on how to install the CERN HEPMC3 module, link to STARlight and compile STARlight with the HEPMC3 option enabled is well described in HEPMC3 Output section of the Github README.md file at <https://github.com/STARlightsim/STARlight/blob/main/README.md>.

Input:

The input parameters are listed below with typical values for LHC Pb-Pb running given in parentheses. Optional parameters are denoted with *.

```
baseFileName          # The name of the output files. STARlight will
                      # copy the input slight.in to baseFileName.in, and
                      # produce output files baseFileName.txt and
                      # baseFileName.out. (slight)

BEAM_1_Z = 82         # Charge of beam one projectile. (82)
BEAM_1_A = 208        # Atomic number of beam one projectile. (208)
BEAM_2_Z = 82         # Charge of beam two projectile. (82)
BEAM_2_A = 208        # Atomic number of beam two projectile. (208)
BEAM_1_GAMMA = 1470   # Lorentz boost for beam one projectile(pz>0).
                      # (1470)
BEAM_2_GAMMA = 1470.0 # Lorentz boost for beam two projectile(pz<0).
                      # (1470)
W_MAX = 12.0          # Maximum value for the gamma-gamma center of mass
                      # energy,  $W = 4E_1E_2$ , in GeV. Setting W_MAX = -1
                      # tells STARlight to use the default value specified
                      # in inputParameters.cpp (recommended for single
                      # meson production). For single mesons, the default
                      # W_MAX is the particle mass plus five times the
                      # width. For lepton pairs, the default W_MAX is
                      # given by  $2\hbar c \sqrt{\frac{Y_1 Y_2}{R_1 R_2}}$ . These are defined in
                      # src/inputParameters.cpp (-1)
W_MIN = -1            # Min value of w. Minimum value for the gamma-gamma
                      # center of mass energy,  $W = 4E_1E_2$ , in GeV. Setting
                      # W_MIN = -1 tells STARlight to use the default
```

value specified in inputParameters.cpp
(recommended for single meson production). The default W_MIN is the larger of the kinematic limit (e.g. $2m_{\pi}$ for decays) or the particle mass minus five times the width. (-1)

W_N_BINS = 40 #Bins w maximum and minimum values for w (the gamma-gamma center of mass energy, $w = 4E_1E_2$), and the number of w bins in the lookup tables (40)

RAP_MAX = 8. # Maximum rapidity of produced particle. (8)

RAP_N_BINS = 80 # Number of rapidity bins used in the cross section calculation (80)

CUT_PT* = 0 # Specifies whether the user chooses to place restrictions on the transverse momentum of the decay products. 0= no, 1 = yes. (0)

PT_MIN* = 1.0 # If a transverse momentum cut is applied, this specifies the minimum value produced, in GeV/c. (1.0)

PT_MAX* = 3.0 # If a transverse momentum cut is applied, this specifies the maximum value produced, in GeV/c. (3.0)

CUT_ETA* = 0 # Specifies whether the user chooses to place restrictions on the pseudorapidity of the decay products. 0= no, 1 = yes. (0). PLEASE NOTE that in the $\omega \rightarrow \pi^+\pi^-\pi^0$ channel Eta cuts are not applied to the uncharged pion. It is only applied to the 2 charged pions.

ETA_MIN* = -10 # If a pseudorapidity cut is applied, this specifies the minimum value produced. (-10)

ETA_MAX* = 10 # If a pseudorapidity cut is applied, this specifies the maximum value produced. (10)

PROD_MODE = 2 #**PROD_MODE=1:** Two-photon interaction.
PROD_MODE=2: Coherent photonuclear vector meson production assuming narrow resonances. This option should also be used for exclusive vector meson production in pp collision. In pA or pp collisions, this option means that the proton emits the photon and that the gamma-A interaction is coherent.
PROD_MODE=3: Coherent photonuclear vector meson production assuming wide resonances. This option should in be used for exclusive ρ^0 production.
PROD_MODE=4: Incoherent photonuclear vector meson production. In pA collisions, this option means that the nucleus emits the photon. Do not use for pp.
PROD_MODE=5: Photonuclear one photon exchange uses DPMJET single.
PROD_MODE=6: Photonuclear two photon exchange (both nuclei excited) uses DPMJET double.
PROD_MODE=7: Photonuclearsinglepa uses DPMJET Single, proton mode.
PROD_MODE=8: [not supported/verified] Photonuclear singlepapy uses Pythia 6

N_EVENTS = 10	#Number of events produced (1000)
PROD_PID = 443013	# For PROD_MODE 1 through 4, this selects the channel to be produced, in PDG notation. Currently supported options are list below. (443013)
RND_SEED = 34533	# Seed for random number generator. (34533)
BREAKUP_MODE = 5	# Specifies the way nuclear break-up is handled. This option only works for lead or gold. It has no meaning in proton-proton or proton-nucleus collisions 1 = hard sphere nuclei (no hadronic break-up if impact parameter is greater than the sum of nuclear radii, no restriction on Coulomb break-up). 2 = requires Coulomb break-up of both nuclei, with no restriction on the number of neutrons emitted by either nucleus (XnXn). 3 = requires Coulomb break-up of both nuclei, but requires that a single neutron is emitted from each nucleus (1n1n). 4 = requires Coulomb break-up of neither nucleus. (0n0n) 5 = requires that there be no hadronic break up, no restriction on Coulomb break-up (This is similar to option 1, but with the actual hadronic interaction probability). 6 = requires Coulomb break up of one or both nuclei, with no restriction on the number of neutrons emitted (XnXn + 0nXn + Xn0n). 7 = requires Coulomb break up of only one nucleus, with no restriction on the number of neutrons emitted (0nXn+ Xn0n). 8 = selectable input parameter range (i.e. for peripheral collisions, not UPCs) regardless of nuclear breakup. Fixed input range between BMAX and BMIN (set by two otherwise optional cards, below)
INTERFERENCE = 0	# Specifies whether interference based on the ambiguity of which nucleus emits the photon is included. The effect of this interference is only visible at very small transverse momentum. 0 = interference off, 1 = interference on. (0)
IF_STRENGTH = 1.	# If interference is turned on, specifies the percentage of interference. The range is -1.0 - 1.0.; 1 is the standard value for ion-ion collisions, while -1.0 is expected for proton-antiproton collisions. (1)
INT_PT_MAX = 0.24	# Used only when the interference option above is turned on. This specifies the maximum transverse momentum considered, in GeV/c. (0.24)
INT_PT_N_BINS = 120	# Used only when the interference option above is turned on. This specifies the number of bins in transverse momentum to use. (120)

INT_PT_WIDTH = 0 #Used only when the interference option above is turned on. This specifies the width of bins in transverse momentum to use. (0)
 XSEC_METHOD* = 0 #Determines which method is used to calculate the cross-section for cross-sections. XSEC_METHOD=0 is faster, but works only for symmetric collisions (i.e. with identical nuclei). XSEC_METHOD=1 always works, but is slower. (0)
 BSLOPE_DEFINITION*=0 Used for proton and nucleon (i. e. incoherent nuclear) collisions to set the t-spectrum, $dN/dt = \exp(-bt)$. When BSLOPE_DEFINITION=1, then the slope is determined by BSLOPE_VALUE (below). When BSLOPE_DEFINITION=2, the slope is calculated as a function of p center of mass energy per the H1 analysis, Eur. Phys. J. C46, 585 (2006):
 $b = 4.63/\text{GeV}^2 + 4 \ln(W_p/90 \text{ GeV})$
 The default value, BSLOPE_DEFINITION=0 has no effect.
 Note that this affects the t-slope only; it does not affect the total cross-section
 BSLOPE_VALUE* WHEN BSLOPE_DEFINITION=1, this determines the exponential slope for $dN/dt = \exp(-BSLOPE_VALUE * t)$
 SELECT_IMPULSE_VM When set =1, performs an impulse approximation calculation (this ignores most nuclear physics, including shadowing). Default=0; no change
 QUANTUM_GLAUBER When set =1, perform a quantum Glauber calculation, rather than classical, which is the default (or when set =0)
 BMIN Needed for Breakup_mode=8. Sets the minimum impact parameter
 BMAX Needed for Breakup mode=8. Sets sthe maximum impact parameter.
 OUTPUT_HEADER Adds a header to the output file. This header will contain various input parameters. (1 for header, 0 for no header, default is no header)
 KEEP_PHI= 0 1= phi is store in output along with other stable particles and its decay daughters are removed from output, 0 = phi is not store in output, while its decay daughter which are stable kept in output(default case of dpmjet).
 Used in starlight + dpmjet production.
 KEEP_KSTAR= 0 1 = K*0 is store in output along with other stable particles and its decay daughters are removed from output, 0 = K*0 is not store in output, while its decay daughter which are stable kept in output(default case of dpmjet).
 Used in starlight + dpmjet production

The physics constants used by STARlight can be set with the following parameters:

deuteronSlopePar	deuteron slope parameter (effective temperature) [(GeV/c) ⁻²]
protonMass	mass of the proton [GeV/c ²]
pionChargedMass	mass of the pi [±] [GeV/c ²]
pionNeutralMass	mass of the pi ⁰ [GeV/c ²]
kaonChargedMass	mass of the K [±] [GeV/c ²]
me	mass of the e [±] [GeV/c ²]
muonMass	mass of the mu [±] [GeV/c ²]
tauMass	mass of the tau [±] [GeV/c ²]
f0Mass	mass of the f ₀ (980) [GeV/c ²]
f0width	width of the f ₀ (980) [GeV/c ²]
f0BrPiPi	branching ratio f ₀ (980) → pi [±] pi [±] and pi ⁰ pi ⁰
etaMass	mass of the eta [GeV/c ²]
etaWidth	width of the eta [GeV/c ²]
etaPrimeMass	mass of the eta' [GeV/c ²]
etaPrimeWidth	width of the eta' [GeV/c ²]
etaCMass	mass of the eta _c [GeV/c ²]
etaCWidth	width of the eta _c [GeV/c ²]
f2Mass	mass of the f ₂ (1270) [GeV/c ²]
f2Width	width of the f ₂ (1270) [GeV/c ²]
f2BrPiPi	[GeV/c] f ₂ (1270) → pi [±] pi [±]
a2Mass	mass of the a ₂ (1320) [GeV/c ²]
a2Width	width of the a ₂ (1320) [GeV/c ²]
f2PrimeMass	mass of the f' ₂ (1525) [GeV/c ²]
f2PrimeWidth	width of the f' ₂ (1525) [GeV/c ²]
f2PrimeBrKK	branching ratio f' ₂ (1525) → K [±] K [±] and K ⁰ K ⁰ _{bar}
zoverz03Mass	mass of four-quark resonance (rho ⁰ pair production) [GeV/c ²]
f0PartialggWidth	partial width f ₀ (980) → g g [GeV/c ²]
etaPartialggWidth	partial width eta → g g [GeV/c ²]
etaPrimePartialggWidth	partial width eta' → g g [GeV/c ²]
etaCPartialggWidth	partial width eta _c → g g [GeV/c ²]
f2PartialggWidth	partial width f ₂ (1270) → g g [GeV/c ²]
a2PartialggWidth	partial width a ₂ (1320) → g g [GeV/c ²]
f2PrimePartialggWidth	partial width f' ₂ (1525) → g g [GeV/c ²]
zoverz03PartialggWidth	partial width four-quark resonance → g g (rho ⁰ pair production) [GeV/c ²]
f0Spin	spin of the f ₀ (980)
etaSpin	spin of the eta
etaPrimeSpin	spin of the eta'
etaCSpin	spin of the eta _c
f2Spin	spin of the f ₂ (1270)
a2Spin	spin of the a ₂ (1320)
f2PrimeSpin	spin of the f' ₂ (1525)
zoverz03Spin	spin of the four-quark resonance → g g (rho ⁰ pair production)
axionSpin	spin of the axion
rho0Mass	mass of the rho ⁰ [GeV/c ²]
rho0Width	width of the rho ⁰ [GeV/c ²]
rho0BrPiPi	branching ratio rho ⁰ → pi [±] pi [±]
rho0PrimeMass	mass of the rho' ⁰ (4 pi [±] final state) [GeV/c ²]

rho0PrimeWidth	width of the rho'^0 (4 pi^+/- final state) [GeV/c^2]
rho0PrimeBrPiPi	branching ratio rho'^0 -> pi^+ pi^-
OmegaMass	mass of the omega [GeV/c^2]
OmegaWidth	width of the omega [GeV/c^2]
OmegaBrPiPi	branching ratio omega -> pi^+ pi^-
PhiMass	mass of the phi [GeV/c^2]
PhiWidth	width of the phi [GeV/c^2]
PhiBrKK	branching ratio phi -> K^+ K^-
JpsiMass	mass of the J/psi [GeV/c^2]
JpsiWidth	width of the J/psi [GeV/c^2]
JpsiBree	branching ratio J/psi -> e^+ e^-
JpsiBrmumu	branching ratio J/psi -> mu^+ mu^-
JpsiBrppbar	branching ratio J/psi -> p pbar
Psi2SMass	mass of the psi(2S) [GeV/c^2]
Psi2SWidth	width of the psi(2S) [GeV/c^2]
Psi2SBree	branching ratio psi(2S) -> e^+ e^-
Psi2SBrmumu	branching ratio psi(2S) -> mu^+ mu^-
Upsilon1SMass	mass of the Upsilon(1S) [GeV/c^2]
Upsilon1SWidth	width of the Upsilon(1S) [GeV/c^2]
Upsilon1SBree	branching ratio Upsilon(1S) -> e^+ e^-
Upsilon1SBrmumu	branching ratio Upsilon(1S) -> mu^+ mu^-
Upsilon2SMass	mass of the Upsilon(2S) [GeV/c^2]
Upsilon2SWidth	width of the Upsilon(2S) [GeV/c^2]
Upsilon2SBree	branching ratio Upsilon(2S) -> e^+ e^-
Upsilon2SBrmumu	branching ratio Upsilon(2S) -> mu^+ mu^-
Upsilon3SMass	mass of the Upsilon(3S) [GeV/c^2]
Upsilon3SWidth	width of the Upsilon(3S) [GeV/c^2]
Upsilon3SBree	branching ratio Upsilon(3S) -> e^+ e^-
Upsilon3SBrmumu	branching ratio Upsilon(3S) -> mu^+ mu^-

The following parameters are used only when interfacing with the PYTHIA and/or DPMJET interfaces:

MIN_GAMMA_ENERGY = 6	#Allows the user to set the minimum photon energy (in GeV) in the rest frame of the target nucleus. The default is 6.0 GeV and it should never be set below this value since DPMJET was not designed to handle low energy interactions.
MAX_GAMMA_ENERGY = 600000	#Allows the user to set the maximum photon energy (in GeV) in the rest frame of the target nucleus. The default is 600000.0 GeV.
PYTHIA_PARAMS = ""	#Used to supply input parameters to the PYTHIA interface. This takes a string to pass on semi-colon separated parameters to PYTHIA 6. eg: "mstj(1)=0;paru(13)=0.1" (the default is a blank string " ")
PYTHIA_FULL_EVENT_RECORD = 1	#Determines whether the full event record from PYTHIA is written to slight.out. true = yes, false = no (false). The additional information added is as follows: daughter production vertex (x [mm], y [mm], z [mm], t [mm/c]), mother1, mother2,

PRINT_VM=2

daughter1, daughter2, PYTHIA particle status code. PYTHIA 8 Particle Properties page describes in more detail the properties of mother, daughter, and status code designations.

This command will make STARlight print out a table of y and $W_{\{\gamma p\}}$, kdN_{γ}/dk and $\sigma(\gamma A)$ for the two possible photon directions. PRINT_VM=0, the assumed default, will not produce this output.

Channels of Interest:

2-Photon Channels

Currently supported 2-photon (prod. mode = 1) channel options:

jetset id	particle

221	eta
331	eta-prime
441	eta-c
9010221	f0(975)
225	f2(1270)
115	a2(1320)
335	f2(1525)
33	rho0 pair
11	e+/e- pair
13	mu+/mu- pair
15	tau+/tau- pair
88	axion-like particle (ALP)

Process 88 refers to the single production of a hypothetical axion-like particle (ALP), which decays to a pair of photons. The ALP mass has to be specified by the user through the parameter AXION_MASS. The narrow width approximation is assumed here, with a fixed axion decay constant of $\Lambda=1$ TeV. (See equation (1) of arXiv:1607.06083 for the appropriate conventions.) The cross section can be then rescaled to arbitrary Λ , as long as the narrow width approximation remains valid.

Pomeron-Photon Channels

Currently supported vector meson (prod. mode = 2/3/4) options:

jetset id	particle

113	rho0
113011	rho0 --> e+e-
113013	rho0 --> mu+mu-
223	omega
223211111	omega --> pi+pi-pi0
333	phi
933	phi _ direct KK
333011	phi ---> e+e-
443011	J/psi --> e+e-
443013	J/Psi --> mu+mu-
4432212	J/psi --> proton antiproton
444011	Psi(2S) --> e+e-

444013	Psi(2S) --> mu+mu-
553011	Upsilon(1S) --> e+e-
553013	Upsilon(1S) --> mu+mu-
554011	Upsilon(2S) --> e+e-
554013	Upsilon(2S) --> mu+mu-
555011	Upsilon(3S) --> e+e-
555013	Upsilon(3S) --> mu+mu-
913	rho0 + direct pi+pi- (with interference). The direct pi+pi- fraction is from the ZEUS results, EPJ C2 p247 (1998)
999	four-prong final states (rho'-like to pi+pi-pi+pi-)

DPMJET:

Simulation of photonuclear interactions with STARlight is possible through an interface with DPMJet. These interfaces can be enabled through options passed to cmake during the configuration process. [Depreciated: Using Pythia 6 as a substitute for DPMJet]

The gfortran compiler is required to use the photonuclear interfaces.

===== 1. Photonuclear interactions with DPMJet =====

----- 1.1. Obtaining and installing DPMJet -----

Download the DPMJet package from here:
<https://github.com/DPMJET/DPMJET/releases>

Unpack the tar file and do

make exe

----- 1.2. Compiling Starlight with DPMJet interface -----

To enable the compilation of the DPMJet interface please follow these steps:

CMake uses an environment variable \$DPMJETDIR to locate the DPMJet object files, so define it.

\$ export DPMJETDIR=<path to dpmjet>

Include the "lib" subdirectory in the path

Then create a build directory for STARlight

\$ mkdir <build-dir>

and change into it

\$ cd <build-dir>

Run CMake with the option to enable DPMJet

\$ cmake <path-to-starlight-source> -DENABLE_DPMJET=ON

Then build it

\$ make

----- 1.3. Running Starlight with DPMJet interface -----

To run Starlight with the DPMJet interface a couple of files are needed in the directory where you want to run Starlight.

The files needed are:

- slight.in** (Starlight config file. An example suitable for DPMJet can be found in config/slight.in.dpmjet)
- my.input** (DPMJet config file. An example can be found in config/my.input)
- dpmjet.dat** (Can be found in the DPMJet source directory)

In addition, create a subdirectory "dpmdata" in the build directory, where Starlight is run from, and copy the files

```
<DPMJetDir>/dpmdata/CT14LL.pds  
<DPMJetDir>/dpmdata/dpmjpar.dat
```

into it.

In the slight.in file the relevant production modes (PROD_MODE) for DPMJET is:

- 5: A+A single excitation
- 6: A+A double excitation
- 7: p+A single excitation

In addition the minimum and maximum gamma energies must be set.

Note that the target A and Z as well as the photon energy range must be the same in slight.in and my.input.

To run:

```
$ ./starlight < my.input
```

```
[DPMJET reads from direct input/interactive]
```

Output

STARlight outputs an ASCII file named slight.out.

If OUTPUT_HEADER = 1 (set in input file), then there will be a header at the beginning of the output file followed by a list of events. If OUTPUT_HEADER = 0, or if OUTPUT_HEADER is not set, then there will be no header in the output file and the file will start with the list of events.

If there is a header, it will be three lines, with the following format:

CONFIG_OPT: prod_mod particle_id nevents q_glauber impulse seed

where prod_mod indicates if a wide or narrow resonance has been used, particle_id specifies the vector meson species (and decay channel) being produced, nevents indicates the total number of events in the simulation, q_glauber indicates if a quantum (=1) or classical (=0) Glauber has been selected, impulse indicates if the nuclear effects are being modelled (=0) or a simple impulse approx. is employed, and finally seed records the random number seed used when initializing the Monte Carlo. The config opt line is followed by two lines with brief descriptions of beams in the collision, with the format:

BEAM_1(2): beam1(2)Z beam1(2)A beam1(2)LorentzGamma

where beam1(2)Z is the charge of the particles in beam 1(2), beam1(2)A indicates the atomic number of beam 1(2) and beam1(2)LorentzGamma is the Lorentz gamma factor associated to beam 1(2)

For each event, a summary line is printed, with the format

EVENT: n ntracks nvertices ,

where n is the event number (starting with 1), ntracks is the number of tracks in the event, and nvertices is the number of vertices in the event (STARlight does not currently produce events with more than one vertex).

EVENT line is followed by a description of the vertex, with the format

VERTEX: x y z t nv nproc nparent ndaughters ,

where x, y, z and t are the 4-vector components of the vertex location, nv is the vertex number, nproc is a number intended to represent physical process (always set to 0), nparent is the track number of parent track (0 for primary vertex) and ndaughters is the number of daughter tracks from this vertex.

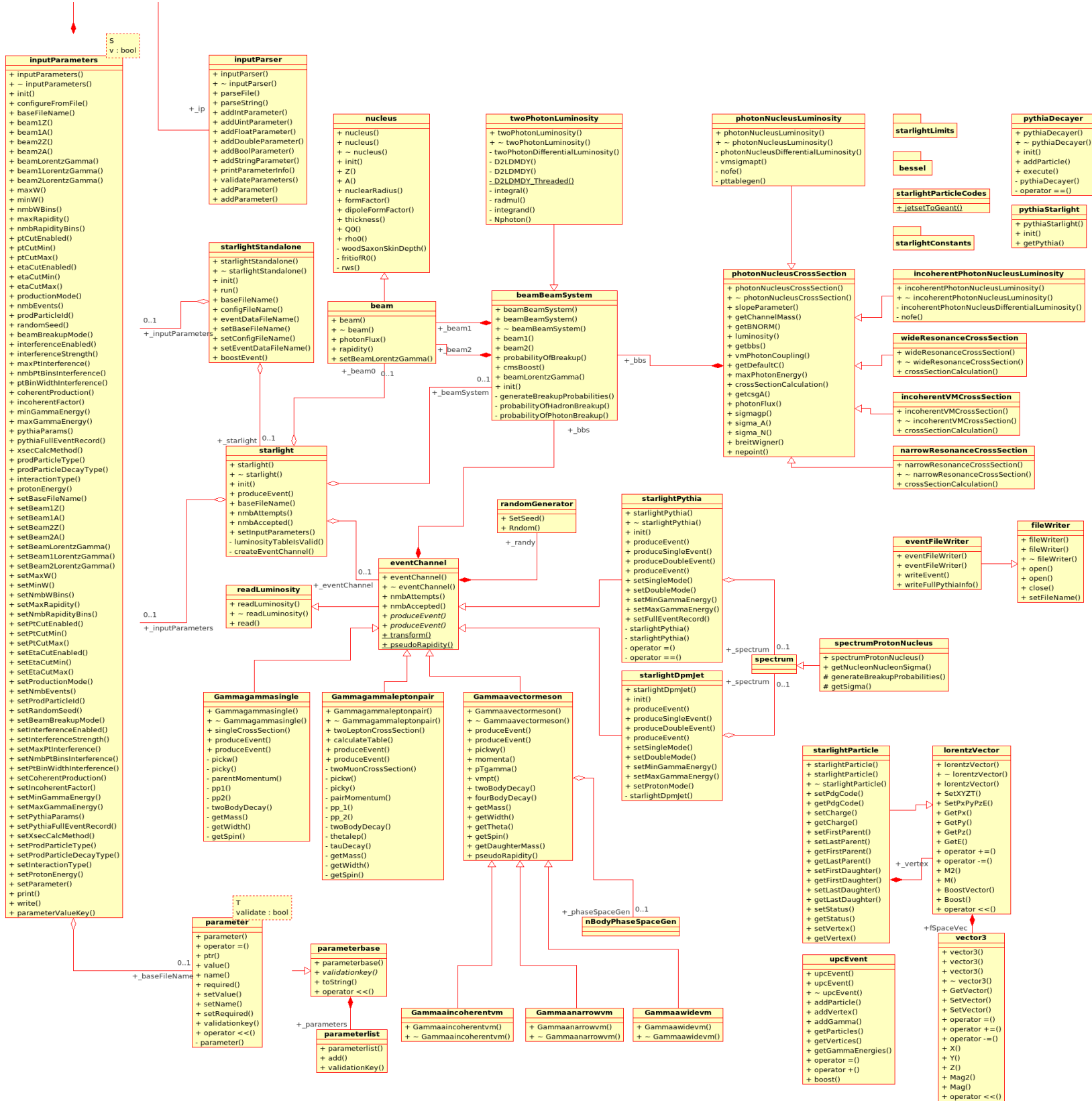
This is followed by a series of lines describing each of the daughter tracks emanating from this vertex. Each track line has the format

TRACK: GPID px py pz nev ntr stopv PDGPID ,

where GPID is the Geant particle id code, px, py and pz are the three vector components of the track's momentum, nev is the event number,

ntr is the number of this track within the vertex (starting with 0), stopv is the vertex number where track ends (0 if track does not terminate within the event), and PDGPID is the Monte Carlo particle ID code endorsed by the Particle Data Group.

Class Diagram



File Descriptions

Readme.pdf

[This file.] provides information on the installation, operation, and construction of STARlight.

CMakeLists.txt

controls STARlight compilation. For details, please see above in . This is the default/supported compilation method.

Makefile

A sample Makefile for compilation on *nix systems. This file is not actively supported. Please use CMake.

starlightconfig.h.in

passes on some compiler settings; such as enabling the Pythia/DPMJet sections within the source code.

starlightDoxyfile.conf

Doxygen configuration file.

CMake Modules:

FindPythia8.cmake

used by CMake to find the Pythia 8 files needed to compile STARlight with Pythia 8 dependent options enabled. It searches for: Pythia.h, Index.xml, libpythia8

FindPythia6.cmake

used by CMake to find the Pythia 6 files needed to compile STARlight with Pythia 6 dependent options enabled. It searches for: libPythia6. *Pythia 6 functionality has been deprecated.*

FindDPMJet.cmake

used by CMake to find the DPMJET files needed to compile STARlight with DPMJET dependent options enabled. It searches for: dpmjet3.0-5.o, pythia6115dpm3v1.o, and phojet1.12-35c4.o

FindROOT.cmake

used by CMake to find the ROOT files needed to compile STARlight with ROOT dependent options enabled. It searches for: root-config. root-config is then used to set the rest of the paths/options needed to enable ROOT within STARlight.

CommonMacros.cmake

A collection of useful cmake macros.

FindLHAPDF.cmake

used by CMake to find the LHAPDF dependent options enabled. This was necessary for older versions of Pythia8, but this is no longer the case. However, this file is being kept in the

distribution for users that would like to re-enable it. It searches for: Pythia.h and liblhpdfdummy

Config files:

my.input

A sample DPMJET configuration file.

slight.in

A sample STARlight input file, to select the desired final state and associated options. The section has more information.

slight.in.dpmjet

A sample slight.in file to use the DPMJET options (eg: PROD_MODE = 5, 6, 7, and MIN_GAMMA_ENERGY, and MAX_GAMMA_ENERGY. and KEEP_PHI = 0,1 and KEEP_KSTAR = 0 ,1).

slight.in.ee_rhic

A sample slight.in file for e+e- production by Au-Au at top RHIC energies

slight.in.jpsi_lhc

A sample slight.in file for J/ production by Pb-Pb at the LHC.

slight.in.pPb_lhc

A sample slight.in file for J/ production by p-Pb at the LHC.

slight.in.rho_rhic

A sample slight.in file for production by Au-Au at top RHIC energies.

dpmjet:

dpmjetint.f

This is a DPMJET library, used in the CMakeLists.txt file to link when enabling DPMJET.

external:

fpe.c

corrects for the floating point trap differences between 32 and 64-bit. The has more information.

pythia6:

pythiainterface.h

interfaces Pythia6 with STARlight. *Pythia 6 functionality has been deprecated.*

utils:

Ana.C

This macro runs Analyze.cxx, which takes as input an ASCII STARlight output file, slight.out, and creates a standard set of histograms, which are stored in histograms.root

Analyze.cxx

This macro reads in a starlight output file and creates histograms of the p_T and rapidity of the daughters, as well as the p_T , rapidity and mass of the parent. It assumes there are only 2 daughter tracks that are electrons, muons, or pions. The histograms for the daughter particles are called fPt2, fPt2, fRap1, and fRap2. Parent histograms are created for each possible daughter species (e.g., parent p_T histograms are created with the names fPtEl, fPtMu, and fPtPi), but only the ones corresponding to the actual daughter particle are filled. The histograms are saved in a file called histograms.root.

To use this Analyze.cxx, modify the file Ana.C to call your input file (as downloaded, it calls slight.out) and the number of events you wish to process (as downloaded, it processes 20 events). Then open root and type ".x Ana.C".

Analyze.h

The header file for Analyze.cxx and Ana.C.

AnalyzeTree.cxx

This macro reads the starlight.root file produced by ConvertStarlightAsciiToTree.C, which contains TLorentzVectors for the parents and a TClonesArray of TLorentzVectors for the daughters. It creates histograms of the p_T and rapidity of the daughters, as well as the p_T , rapidity and mass of the parent. While the parents may have been created as the vector sum of any number of daughter particles, this macro currently produces histograms for only the first two daughter particles. The daughter histograms are called D1Pt, D2Pt, D1Rapidity, and D1Rapidity. Parent histograms are named ParentPt, ParentRapidity, and ParentMass. The histograms are stored in starlight_histos.root.

To use Analyzetree.cxx, first run ConvertStarlightAsciiToTree.C to produce the starlight.root file. If needed, modify the file AnalyzeTree.h to call your input file (as downloaded, it calls starlight.root). Then open root and type .x AnaTree.C .

AnalyzeTree.h

The header file for AnalyzeTree.cxx.

AnaTree.C

compiles and runs AnalyzeTree.cxx, which takes as input the starlight.root file produced by ConvertStarlightAsciiToTree.cxx output histograms are stored in starlight_histos.root

ConvertStarlightAsciiToTree.C

reads a starlight output file (default name slight.out) and creates a root file with TLorentzVectors for the parent and a TClonesArray of TLorentzVectors for the daughter particles. The output is stored in a root file (default name starlight.root) with one branch labeled "parent" and the other labeled "daughters". Any number of daughter tracks can be accommodated. Daughter species currently accommodated are: electrons, muons, charged or neutral pions, charged or neutral kaons, and protons.

To use AnaTree.C, open root and then type `.x ConvertStarlightAsciiToTree.C("inputfilename", "outputfilename")` The root file produced can be examined in a root TBrowser.

A macro to read this root file and make some standard plots is also provided. This macro is called AnalyzeTree.cxx; it can be compiled and run with the AnaTree.C macro by opening root and typing `.x AnaTree.C()`

Source Files:

beam.cpp

generates the beam class, which inherits from the nucleus class (cf.). The object represents an accelerated nucleus, or a beam.

Functions:

```
beam::beam
beam::~~beam
beam::photonFlux // calculates the "photon density" given
                  the impact parameter and energy.
```

beambeamssystem.cpp

represents the colliding system of interest.

Functions:

```
beamBeamSystem::beamBeamSystem
beamBeamSystem::~~beamBeamSystem
beamBeamSystem::probabilityOfBreakup
beamBeamSystem::generateBreakupProbabilities
beamBeamSystem::probabilityOfHadronBreakup
beamBeamSystem::probabilityOfPhotonBreakup
```

bessel.cpp

calculate modified Bessel functions of the first and second kind.

Functions:

```
bessel::besI0
bessel::dbesk0
bessel::dbesk1
bessel::besI1
```

eventchannel.cpp

inherits from readLuminosity. It is a base for class for functions to produce events that is overloaded by other classes (Gammagammaleptonpair, Gammagammasingle, Gammaavectormeson, starlightDpmJet, and starlightPythia).

Functions:

```
eventChannel::eventChannel
eventChannel::~~eventChannel
eventChannel::transform // Lorentz Tranforms the frame
eventChannel::pseudoRapidity // calculates the
                             pseudorapidity with the input from px, py, and pz
```

eventfilewriter.cpp

writes event information in the output file.

Functions:

```
eventFileWriter::eventFileWriter
eventFileWriter::~~eventFileWriter
eventFileWriter::writeEvent
```

filewriter.cpp

The base class for eventFileWriter, which is writes event information in the output file.

Functions:

```
fileWriter::fileWriter()
fileWriter::~~fileWriter()
fileWriter::open
fileWriter::open(filename)
fileWriter::close
```

gammaaluminosity.cpp

contains the photonNucleusLuminosity class, which inherits from photonNucleusCrossSection. It calculates the differential cross-section for gamma-A interactions.

Functions:

```
photonNucleusLuminosity::photonNucleusLuminosity
photonNucleusLuminosity::~~photonNucleusLuminosity
photonNucleusLuminosity::photonNucleusDifferentialLuminosity //Calculates and outputs the differential luminosity
photonNucleusLuminosity::pttablegen // Calculates the pt spectra for VM production with interference per S. Klein and J. Nystrand, Phys. Rev Lett. 84, 2330 (2000).
photonNucleusLuminosity::vmsigmapt //calculates th effect of the nuclear form factor on the pt spectrum, for use in interference calculations. It calculates the cross section suppression SIGMA_PT(P_T) as a function of p_t. The input p_t values come from ptable.inc
photonNucleusLuminosity::nofe //calculates the 'photon density' $d^2N_{\gamma}/db^2$ 
```

gammaavm.cpp

is responsible for classes Gammaavectormeson, Gammaanarrowvm, and Gammaawidevm. Both Gammaanarrowvm and Gammaawidevm inherit from Gammaavectormeson, which inherits from eventChannel. The classes are responsible for generating and decaying the vector mesons produced by photon-nucleus interactions.

Functions:

```
Gammaaavectormeson::Gammaaavectormeson
Gammaaavectormeson::~~Gammaaavectormeson
Gammaaavectormeson::pickwy //responsible for selecting the
events center of mass energy and rapidity
Gammaaavectormeson::twoBodyDecay // This routine decays a
particle into two particles of mass mdec, taking spin into
account
Gammaaavectormeson::fourBodyDecay // decays a particle into
four particles with isotropic angular distribution
Gammaaavectormeson::getDaughterMass //returns the daughter
particles mass, & the final particles id...
Gammaaavectormeson::getTheta //This depends on the decay
angular distribution
Gammaaavectormeson::getWidth
Gammaaavectormeson::getMass
Gammaaavectormeson::getSpin //it's a VM, returns 1
Gammaaavectormeson::momenta // calculates momentum and
energy of vector meson given W and Y, without
interference.
Gammaaavectormeson::pTgamma //finds the photon pT
Gammaaavectormeson::vmpt // calculates momentum and energy
of a vector meson given W and Y, including interference.
It gets the pt distribution from a lookup table.
produceEvent
pseudorapidity
Gammaanarrowvm::Gammaanarrowvm
Gammaanarrowvm::~~Gammaanarrowvm
Gammaanarrowvm::gammaaincoherentvm
Gammaawidevm::Gammaawidevm
Gammaawidevm::~~Gammaawidevm
```

gammagammaleptonpair.cpp

inherits from eventChannel. It calculates the lepton pair's cross-section and generates and decays the lepton pairs.

Functions:

```
Gammagammaleptonpair::Gammagammaleptonpair
Gammagammaleptonpair::~~Gammagammaleptonpair
Gammagammaleptonpair::twoLeptonCrossSection // calculates
section for 2-particle decay, per, see STAR Note 243, Eq.
9. It calculates the 2-lepton differential cross section
Gammagammaleptonpair::twoMuonCrossSection // gives the
two muon cross section as a function of Y&W, per G.Soff
et. al Nuclear Equation of State, part B, 579
Gammagammaleptonpair::pickw // Picks a w for the 2- photon
calculation.
Gammagammaleptonpair::picky // Picks a y given a W
Gammagammaleptonpair::pairMomentum // calculates
px,py,pz,and E given w and y
Gammagammaleptonpair::pp_1 // For beam 1, returns a
random momentum drawn from from pp_1(E) distribution
Gammagammaleptonpair::pp_2 // For beam 2, returns a
random momentum drawn from from pp_2(E) distribution
Gammagammaleptonpair::twoBodyDecay //decays a particle
into two particles of mass mdec, taking spin into account
```

```

Gammagammaleptonpair::thetalep // calculates the cross-
section as a function of angle for a given W and Y, for
the production of two muons or taus, per Brodsky et al.
PRD 1971, 1532 equation 5.7
Gammagammaleptonpair::produceEvent //returns the vector
with the decay particles inside
Gammagammaleptonpair::calculateTable //calculates the
tables that are used elsewhere in the Monte Carlo the tau
decay follows V-A theory,  $1 - 1/3 \cos(\theta)$  the energy of
each of the two leptons in tau decay is calculated using
formula 10.35 in "Introduction to elementary particles by
D. Griffiths," which assumes that the mass of the electron
is 0. The maximum electron energy in in such a system is
 $0.5 * \text{mass of the tau}$ 
Gammagammaleptonpair::tauDecay // assumes that the
tauons decay to electrons and calculates the directons of
the decays
Gammagammaleptonpair::getMass
Gammagammaleptonpair::getWidth
Gammagammaleptonpair::getSpin

```

gammagammasingle.cpp

inherits from eventChannel. It calculates the cross-section for single mesons and generates and decays the single mesons from gamma-gamma interactions. It also generates single mesons which are then decayed by Pythia 8.

Functions:

```

Gammagammasingle::Gammagammasingle
Gammagammasingle::~~Gammagammasingle
Gammagammasingle::singleCrossSection // calculates the
cross-section in the narrow-width approximation, per STAR
Note 243, Eq. 8
Gammagammasingle::pickw // picks a w for the 2-photon
calculation.
Gammagammasingle::picky
Gammagammasingle::parentMomentum // calculates
px,py,pz, and E given w and y
Gammagammasingle::pp_1 // For beam 1, returns a random
momentum drawn from from pp(E) distribution
Gammagammasingle::pp_2 // For beam 2, returns a random
momentum drawn from from pp(E) distribution
Gammagammasingle::twoBodyDecay //decays a particle into
two particles of mass mdec, taking spin into account
Gammagammasingle::produceEvent
Gammagammasingle::getMass
Gammagammasingle::getSpin

```

incoherentPhotonNucleusLuminosity.cpp

is responsible for the incoherentPhotonNucleusLuminosity class and inherits from photonNucleusCrossSection. It houses the differential luminosity calculation for incoherent gamma-A interactions.

Functions:

```

incoherentPhotonNucleusLuminosity::incoherentPhotonNucleus
Luminosity

```

```

incoherentPhotonNucleusLuminosity::~incoherentPhotonNucleu
sLuminosity
incoherentPhotonNucleusLuminosity::incoherentPhotonNucleus
DifferentialLuminosity
incoherentPhotonNucleusLuminosity::nofe //Function for the
calculation of the "photon density".

```

incoherentVMCrossSection.cpp

inherits from photonNucleusCrossSection. It calculates the cross-section for incoherent photon-nucleus interactions.

Functions:

```

incoherentVMCrossSection::incoherentVMCrossSection
incoherentVMCrossSection::~incoherentVMCrossSection
incoherentVMCrossSection::crossSectionCalculation //
calculates the vector meson cross section assuming a
narrow resonance. For reference, see STAR Note 386.

```

inputParameters.cpp

sets and stores STARlight's input parameters.

Functions:

```

inputParameters::inputParameters
inputParameters::~inputParameters
inputParameters::init
inputParameters::configureFromFile
inputParameters::print
inputParameters::write
inputParameters::parameterValueKey

```

inputParser.cpp

parses the input files and stores the information in the inputParameters.

Functions:

```

inputParser::inputParser()
inputParser::~inputParser()
inputParser::parseFile
inputParser::parseString
inputParser::addIntParameter
inputParser::addUIntParameter
inputParser::addFloatParameter
inputParser::addDoubleParameter
inputParser::addBoolParameter
inputParser::addStringParameter
inputParser::printParameterInfo
inputParser::validateParameters

```

lorentzvector.cpp

holds Lorentz 4-vectors.

Functions:

```

lorentzVector::lorentzVector
lorentzVector::~lorentzVector
SetXYZT

```

main.cpp

the "main" file/function—where the program starts.

narrowResonanceCrossSection.cpp

inherits from photonNucleusCrossSection. It calculates the cross-section for narrow resonance vector mesons.

Functions:

```
narrowResonanceCrossSection::narrowResonanceCrossSection  
narrowResonanceCrossSection::~~narrowResonanceCrossSection  
narrowResonanceCrossSection::crossSectionCalculation //  
calculates the vector meson cross section assuming a  
narrow resonance, per STAR Note 386.
```

nBodyPhaseSpaceGen.cpp

is responsible for the kinematics used in the four-prong decays.

Functions:

```
nBodyPhaseSpaceGen::nBodyPhaseSpaceGen  
nBodyPhaseSpaceGen::~~nBodyPhaseSpaceGen  
nBodyPhaseSpaceGen::setDecay // sets decay constants and  
prepares internal variables  
nBodyPhaseSpaceGen::generateDecay// generates event with  
certain n-body mass and momentum and returns event weight  
general purpose function  
nBodyPhaseSpaceGen::generateDecayAccepted// generates full  
event with certain n-body mass and momentum only, when  
event is accepted (return value = true) this function is  
more efficient, if only weighted events are needed  
nBodyPhaseSpaceGen::pickMasses// randomly chooses the (n -  
2) effective masses of the respective (i + 1)-body systems  
nBodyPhaseSpaceGen::calcWeight// computes event weight (=   
integrand value) and breakup momenta uses vector of  
intermediate two-body masses prepared by pickMasses()  
nBodyPhaseSpaceGen::calcEventKinematics// calculates  
complete event from the effective masses of the (i + 1)-  
body systems, the Lorentz vector of the decaying system,  
and the decay angles uses the break-up momenta calculated  
by calcWeight()  
nBodyPhaseSpaceGen::estimateMaxWeight// calculates maximum  
weight for given n-body mass  
nBodyPhaseSpaceGen::print
```

nucleus.cpp

defines the basic properties of a nucleus such as radius, form factor, and thickness.

Functions:

```
nucleus::nucleus  
nucleus::~~nucleus  
nucleus::init  
nucleus::nuclearRadius  
nucleus::formFactor  
nucleus::dipoleFormFactor  
nucleus::thickness// calculates the nuclear thickness  
function per Eq. 4 in Klein and Nystrand, PRC 60
```

photonNucleusCrossSection.cpp

calculates the cross-section for coherent photon-Nucleus interactions.

Functions:

```

photonNucleusCrossSection::photonNucleusCrossSection
photonNucleusCrossSection::~~photonNucleusCrossSection
photonNucleusCrossSection::getcsgA // returns the cross-
section for photon-nucleus interaction producing vector
mesons
photonNucleusCrossSection::photonFlux // gives the
photon flux as a function of energy Egamma for arbitrary
nuclei and gamma. The first time it is called, it
calculates a lookup table which is used on subsequent
calls. It returns dn_gamma/dE (dimensions 1/E), not dI/dE
energies are in GeV, in the lab frame
photonNucleusCrossSection::nepoint// gives the spectrum of
virtual photons, dn/dEgamma, for a point charge q=Ze
sweeping past the origin with velocity gamma, integrated
over impact parameter from bmin to infinity, per Eq. 15.54
of Jacksons Classical Electrodynamics
photonNucleusCrossSection::sigmagp// gives the gamma-
proton --> VectorMeson cross section. Wgp is the gamma-
proton CM energy. Unit for cross section: fm**2
photonNucleusCrossSection::sigma_A// Nuclear Cross Section
sig_N,sigma_A in (fm**2)
photonNucleusCrossSection::sigma_N// Nucleon Cross Section
in (fm**2)
photonNucleusCrossSection::breitWigner// uses simple
fixed-width s-wave Breit-Wigner without coherent
background for rho' (PDG '08 eq. 38.56)

```

pythiadecayer.cpp

links Pythia 8 and STARlight, and initializes Pythia 8.

Functions:

```

pythiaDecayer::pythiaDecayer
pythiaDecayer::~~pythiaDecayer
pythiaDecayer::init
pythiaDecayer::addParticle
pythiaDecayer::execute

```

randomgenerator.cpp

STARlight's random number generator, using the same algorithm as ROOTs TRANDOM3 class. It is based on M. Matsumoto and T. Nishimura, Mersenne Twistor: A 623-dimensionally equidistributed uniform pseudorandom number generator. For more information see

<http://www.math.keio.ac.jp/~matumoto/emt.html>

Functions:

```

randomGenerator::SetSeed
randomGenerator::Rndom

```

readinluminosity.cpp

reads in the luminosity tables from slight.txt, which is generated in the early stages of the program.

Functions:

```

readLuminosity::readLuminosity
readLuminosity::~~readLuminosity
readLuminosity::read

```

spectrum.cpp

sets up functions needed to make cross-section calculations for general photonuclear interactions modeled with DPMJET.

Functions:

- spectrum::spectrum
- spectrum::generateKsingle
- spectrum::generateKdouble
- spectrum::drawKsingle
- spectrum::drawKdouble
- spectrum::generateBreakupProbabilities
- spectrum::getFnSingle
- spectrum::getFnDouble
- spectrum::getTransformedNofe

spectrumprotonnucleus.cpp

sets up functions needed to make cross-section calculations for general photonuclear interactions modeled with DPMJET.

Functions:

- spectrumProtonNucleus::spectrumProtonNucleus
- spectrumProtonNucleus::generateBreakupProbabilities
- spectrumProtonNucleus::getSigma

starlight.cpp

initializes and then produces and decays events.

Functions:

- starlight::starlight
- starlight::~~starlight
- starlight::init
- starlight::produceEvent
- starlight::luminosityTableIsValid
- starlight::createEventChannel

starlightdpmjet.cpp

hosts the class starlightDpmJet which inherits from the eventChannel class. It includes methods to generate diffractive events with DPMJET.

Functions:

- starlightDpmJet::starlightDpmJet
- starlightDpmJet::init
- starlightDpmJet::produceEvent
- starlightDpmJet::produceSingleEvent
- starlightDpmJet::produceDoubleEvent

starlightparticle.cpp

is a container to store particle information.

Functions:

- starlightParticle::starlightParticle
- starlightParticle::~~starlightParticle

starlightparticlecodes.cpp

converts jetset particle numbers to the corresponding GEANT code.

Functions:

- starlightParticleCodes::jetsetToGeant

starlightpythia.cpp

inherits from the eventChannel class. It includes methods to calculate diffractive events with Pythia6. *Pythia 6 functionality has been deprecated.*

Functions:

- starlightPythia::starlightPythia
- starlightPythia::~~starlightPythia
- starlightPythia::init
- starlightPythia::produceEvent

starlightStandalone.cpp

is used by Main.cpp and in turn calls methods from the starlight class.

Functions:

- starlightStandalone::starlightStandalone
- starlightStandalone::~~starlightStandalone
- starlightStandalone::init
- starlightStandalone::run
- starlightStandalone::boostEvent

twophotonluminosity.cpp

inherits from beamBeamSystem, and is responsible for calculating the two photon luminosity table based on W and Y.

Functions:

- twoPhotonLuminosity::twoPhotonLuminosity
- twoPhotonLuminosity::~~twoPhotonLuminosity
- twoPhotonDifferentialLuminosity
- twoPhotonLuminosity::D2LDMDY
- twoPhotonLuminosity::D2LDMDY_Threated
- twoPhotonLuminosity::integral
- twoPhotonLuminosity::radmul
- twoPhotonLuminosity::integrand
- twoPhotonLuminosity::Nphoton

upcevent.cpp

stores the final event information.

Functions:

- upcEvent::upcEvent
- upcEvent::operator=
- upcEvent::operator+
- upcEvent::boost

vector3.cpp

is a container for 3D-vectors.

Functions:

- vector3::vector3
- vector3::~~vector3
- vector3::SetVector

wideResonanceCrossSection.cpp

inherits from photnNucleusCrossSection. It is responsible for calculating the cross-section of vector mesons with a wide resonance (eg. Rho).

Functions:

- wideResonanceCrossSection::wideResonanceCrossSection
- wideResonanceCrossSection::~~wideResonanceCrossSection

```
wideResonanceCrossSection::crossSectionCalculation //  
calculates the cross-section assuming a wide(Breit-Wigner)  
resonance.
```

Include Files:

```
beam.h //This class includes a single beam of nucleons  
Included in files
```

Functions

```
beam  
~beam  
rapidity  
photonFlux  
setBeamLorentzGamma
```

```
beambeamsystem.h //This class covers a coliding beam system  
Included in files
```

Functions

```
beamBeamSystem  
~beamBeamSystem  
cmsBoost  
beamLorentzGamma  
beam1  
beam2  
probabilityOfBreakup  
init  
generateBreakupProbabilities  
probabilityOfHadronBreakup  
probabilityOfPhotonBreakup
```

```
bessel.h  
Included in files
```

Functions

besI0
dbesk0
dbesk1
besI1

eventchannel.h

Included in files

Functions

eventChannel
~eventChannel
nmbAttempts ///< returns number of attempted events
nmbAccepted ///< returns number of accepted events
produceEvent
transform ///< Lorentz-transforms given 4-vector
pseudoRapidity ///< calculates pseudorapidity for
given 3-momentum

eventfilewriter.h

Included in files

Functions

eventFileWriter
writeEvent /** Write an UPC event to file */
writeFullPythiaInfo /** Set if we want to write full
pythia information */

filewriter.h

Included in files

Functions

fileWriter
~fileWriter
open //opens the file
setFileName//set the filename we're writing to

gammaaluminosity.h

Included in files

Functions

photonNucleusLuminosity
~photonNucleusLuminosity
photonNucleusDifferentialLuminosity
vmsigmapt
nofe
pttablegen

gammaavm.h

Included in files

Functions

Gammaavectormeson
~Gammaavectormeson
produceEvent
pickwy
momenta
pTgamma
vmpt
twoBodyDecay
fourBodyDecay
getMass
getWidth
getTheta
getSpin
getDaughterMass
pseudoRapidity
Gammaanarrowvm
~Gammaanarrowvm
Gammaawidevm
~Gammaawidevm
Gammaaincoherentvm
~Gammaaincoherentvm

gammagammaleptonpair.h

Included in files

Functions

Gammagammaleptonpair
~Gammagammaleptonpair
twoLeptonCrossSection
calculateTable
produceEvent
twoMuonCrossSection
pickw
picky
pairMomentum
pp_1
pp_2
twoBodyDecay

thetalep
tauDecay
getMass
getWidth
getSpin

gammagammasingle.h

Included in files

Functions

Gammagammasingle
~Gammagammasingle
singleCrossSection
produceEvent
pickw
picky
parentMomentum
pp
twoBodyDecay
thephi
getMass
getWidth
getSpin

incoherentPhotonNucleusLuminosity.h

Included in files

Functions

incoherentPhotonNucleusLuminosity
~incoherentPhotonNucleusLuminosity
incoherentPhotonNucleusDifferentialLuminosity
nofe

incoherentVMCrossSection.h

Included in files

Functions

incoherentVMCrossSection
~incoherentVMCrossSection
crossSectionCalculation

inputParameters.h

Included in files

Functions

parameterlist
add
validationKey
parameterbase
toString
operator<<
parameter
operator=
ptr
value
name
required
setValue
setName
setRequired
inputParameters
~inputParameters
init
configureFromFile
baseFileName
beam1Z
beam1A
beam2Z
beam2A
beamLorentzGamma
beam1LorentzGamma
beam2LorentzGamma
maxW
minW
nmbWBins
MaxRapidity
phiSwitch
kstarSwitch
nmbRapidityBins
ptCutEnabled
ptCutMin
ptCutMax
etaCutEnabled
etaCutMin
etaCutMax
productionMode
nmbEvents
prodParticleId
randomSeed
beamBreakupMode
interferenceEnabled
interferenceStrength
maxPtInterference
nmbPtBinsInterference
ptBinWidthInterference
coherentProduction

incoherentFactor
minGammaEnergy
maxGammaEnergy
pythiaParams
pythiaFullEventRecord
xsecCalcMethod
prodParticleType
prodParticleDecayType
interactionType
protonEnergy
setBaseFileName
setBeam1Z
setBeam1A
setBeam2Z
setBeam2A
setBeamLorentzGamma
setBeam1LorentzGamma
setBeam2LorentzGamma
setMaxW
setMinW
setNmbWBins
SetMaxRapidity
setphiSwitch
setkstarSwitch
setNmbRapidityBins
setPtCutEnabled
setPtCutMin
setPtCutMax
setEtaCutEnabled
setEtaCutMin
setEtaCutMax
setProductionMode
setNmbEvents
setProdParticleId
setRandomSeed
setBeamBreakupMode
setInterferenceEnabled
setInterferenceStrength
setMaxPtInterference
setNmbPtBinsInterference
setPtBinWidthInterference
setCoherentProduction
setIncoherentFactor
setMinGammaEnergy
setMaxGammaEnergy
setPythiaParams
setPythiaFullEventRecord
setXsecCalcMethod
setProdParticleType
setProdParticleDecayType
setInteractionType
setProtonEnergy
setParameter
print
write
parameterValueKey
instance

inputParser.h
Included in files

Functions

inputParser
inputParser
parseFile/** Parse a file */
parseString
addIntParameter
addUIntParameter
addFloatParameter
addDoubleParameter
addBoolParameter
addStringParameter
printParameterInfo
validateParameters
_parameter
operator==
operator<
printParameterInfo
addParameter

lorentzvector.h
Included in files

Functions

lorentzVector
~lorentzVector
SetXYZT
SetPxPyPzE
GetPx
GetPy
GetPz
GetE
operator +=
operator -=
M2
M
BoostVector
Boost
operator <<

narrowResonanceCrossSection.h
Included in files

Functions

narrowResonanceCrossSection
~narrowResonanceCrossSection
crossSectionCalculation

nBodyPhaseSpaceGen.h
Included in files

Functions

Factorial
breakupMomentum
nBodyPhaseSpaceGen
~nBodyPhaseSpaceGen
setDecay
random
generateDecay
generateDecayAccepted
setMaxWeight
maxWeight
normalization
eventWeight
maxWeightObserved
resetMaxWeightObserved
estimateMaxWeight
eventAccepted
daughter
daughters
nmbOfDaughters
daughterMass
intermediateMass
breakupMom
cosTheta
phi
print
operator <<
pickMasses
calcWeight
pickAngles
calcEventKinematics
eventAccepted

nucleus.h
Included in files

Functions

nucleus
~nucleus
init
Z
A
nuclearRadius

```

formFactor
dipoleFormFactor
thickness
Q0
rho0
woodSaxonSkinDepth
fritiofR0
rws

```

photonNucleusCrossSection.h

Included in files

Functions

```

photonNucleusCrossSection
~photonNucleusCrossSection
slopeParameter///< returns slope of t-distribution
[(GeV/c)^{-2}]
getChannelMass ///< returns mass of the produced
system [GeV/c^2]
getBNORM
luminosity///< returns luminosity [10^{26} cm^{-2}
sec^{-1}]
getbbs///< returns beamBeamSystem
vmPhotonCoupling ///< vectormeson-photon coupling
constant f_v / 4 pi (cf. Eq. 10 in KN PRC 60 (1999)
014903)
getDefaultC
maxPhotonEnergy///< returns max photon energy in lab
frame [GeV] (for vectormesons only)
crossSectionCalculation
getcsgA
photonFlux
sigmagp
sigma_A
sigma_N
breitWigner
nepoint

```

pythiadecayer.h

Included in files

Functions

```

pythiaDecayer
~pythiaDecayer
init// Initialize
addParticle// Add particle to current event
execute// Execute event and return starlight type
event
pythiaDecayer

```

operator==

PythiaStarlight.h

Included in files

Functions

pythiaStarlight
init
getPythia

randomgenerator.h

Included in files

Functions

SetSeed
Rndom
randomGenerator
instance

readinluminosity.h

Included in files

Functions

readLuminosity
~readLuminosity
read

reportingUtils.h

Included in files

Functions

getClassMethod__
printErr
printWarn
printInfo
svnVersion

```

printSvnVersion
compileDir
printCompilerInfo
operator <<
progressIndicator
trueFalse
yesNo
onOff
enDisabled

```

spectrum.h

Included in files

Functions

```

spectrum // Spectrum must be constructed with beam-
beam system, default constructor disallowed
generateKsingle // Generate a table of photon energy
probabilities. Use NK+1 logarithmic steps between
Et_min and Eg_max
generateKdouble // Generate a 2-D table of photon
energy probabilities. Use NK+1 x NK+1 logarithmic
steps between Et_min and Eg_max
drawKsingle // Get the energy of a single gamma
@return energy of the gamma
drawKdouble // Get the energy of a single gamma
@param egamma1 variable passed by reference to get
the energy of the first gamma @param egamma2 variable
passed by reference to get the energy of the second
gamma @return energy of the gamma
setBeamBeamSystem // Set the beam beam system
setMinGammaEnergy //Set the minimum gamma energy
setMaxGammaEnergy / Set the maximum gamma energy
setBmin //Set minimum impact parameter
setBMax //Set maximum impact parameter
generateBreakupProbabilities //Generate the hadron
breakup probability table
getSigma ---1.05?
getTransformedNofe
getFnSingle
getFnDouble

```

spectrumprotonnucleus.h

Included in files

```

spectrumprotonnucleus.cpp
starlightdpmjet.cpp
starlightpythia.cpp

```

Functions

```

spectrumProtonNucleus
getNucleonNucleonSigma --- 7.35?
generateBreakupProbabilities
getSigma

```

starlight.h

Included in files

Functions

starlight
~starlight
init
produceEvent
configFileName
nmbAttempts
nmbAccepted
luminosityTableIsValid
createEventChannel

starlightconstants.h

Included in files

Functions

N/A

starlightdpmjet.h

Included in files

Functions

starlightDpmJet
init
produceEvent
produceSingleEvent
produceDoubleEvent
setSingleMode
setDoubleMode
setMinGammaEnergy
setMaxGammaEnergy

setProtonMode

starlightlimits.h

Included in files

Functions

N/A

starlightparticle.h

Included in files

Functions

starlightParticle
~starlightParticle
setPdgCode
getPdgCode
setCharge
getCharge
setFirstParent
getFirstParent
setLastParent
getLastParent
setFirstDaughter
getFirstDaughter
setLastDaughter
getLastDaughter
getStatus
setStatus
setVertex
getVertex

starlightparticlecodes.h

Included in files

Functions

jetsetToGeant//Converts a jetset code into a GEANT
codes

starlightpythia.h

Included in files

Functions

starlightPythia
~starlightPythia
init
produceSingleEvent
produceDoubleEvent
produceEvent
setSingleMode
setDoubleMode

setMinGammaEnergy
setMaxGammaEnergy
setFullEventRecord

starlightStandalone.h
Included in files

Functions

starlightStandalone
~starlightStandalone
init
run
configFileName
eventDataFileName
setConfigFileName
setEventDataFileName
boostEvent

twophotonluminosity.h
Included in files

Functions

twoPhotonLuminosity
~twoPhotonLuminosity
twoPhotonDifferentialLuminosity
D2LDMDY
D2LDMDY_Threaded
integral
radmul
integrand
Nphoton

upcevent.h
Included in files

Functions

upcEvent
~upcEvent
addParticle
addVertex
addGamma
getParticles
getVertices
getGammaEnergies
operator=
operator+
boost

vector3.h

Included in files

Functions

vector3
~vector3
GetVector
SetVector
operator +=
operator =
operator -=
X
Y
Z
Mag2
Mag
operator <<

wideResonanceCrossSection.h

Included in files

Functions

wideResonanceCrossSection
~wideResonanceCrossSection
crossSectionCalculation